

Enoncés TDs et TPs / Angular (partie 2)

1. TD formulaire de login (en mode signalForm)

A partir de la version 21, angular offre une nouvelle manière de gérer les contrôles de saisies dans les formulaires : le mode **signalForm** (mieux que templateDriven et ancien reactiveForm)

Pour **bien organiser le code**, créer *dans src/app* un **nouveau "folder"** `src/app/common` et un **sous répertoire** `src/app/common/data`

Après s'être placé dans `src/app/common/data` (via `cd`), générer un **nouveau fichier** `login.ts`

`src/app/common/data/login.ts`

```
export interface LoginData {
  username: string;
  password: string;
  roles:string;
}

export class Login implements LoginData {
  constructor(
    public username : string = "",
    public password : string = "",
    public roles : string = "" ){}
}
```

Au sein de la classe **LoginComponent**, ajouter :

- un **attribut** `loginModel` initialisé par `= signal<LoginData>(new Login());`
- un attribut `loginForm = form(this.loginModel);`

Ajouter également dans LoginComponent (coté .ts):

- une propriété **message** de type **string**
- une méthode **onLogin()** qui dans un premier temps se contentera de mettre à jour le message en fonction des valeurs saisies cotés .html :

`src/app/login/login.component.ts`

```
import { Component, OnInit } from '@angular/core';
import { Login } from '../common/data/login'

@Component({
  selector: 'app-login',
  imports: [FormsModule],
  templateUrl: './login.component.html',
  styleUrls: ['./login.component.scss']
})
export class LoginComponent {

  loginModel = signal<LoginData>(new Login());
  loginForm = form(this.loginModel);

  public message /* :string */ = "";
  public onLogin(){
    this.message = "donnees saisies = " + JSON.stringify(this.loginModel());
  }
}
```

```
}
```

Dans **LoginComponent.html** ajouter de quoi saisir username, password et roles avec des **[formField]** paramétrés sur **loginForm.username** etc :

Ajouter également un bouton poussoir **login** déclenchant **onLogin()**:

src/app/login/login.component.html

```
<h3>login</h3>

<form>
  <label class="basic-align">username:</label>
  <input [formField]="loginForm.username" /> <br/>

  <label class="basic-align">password:</label>
  <input [formField]="loginForm.password" /><br/>

  <label class="basic-align">roles:</label>
  <input [formField]="loginForm.roles" /><br/>

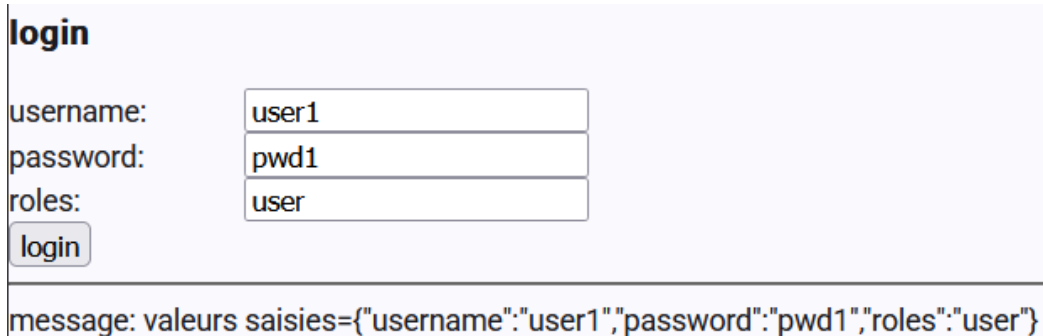
  <input type="button" value="login" (click)="onLogin()" />
</form>
<hr/>
message: <span>{{message}}</span>
```

Dans un premier temps , on pourra se contenter d'un simple alignement de ce genre du côté .css

```
label.basic-align { width : 8em; display: inline-block;}
```

Lancer **ng serve** dans un terminal libre de *Visual Studio Code*

Visualiser un résultat de ce type dans un navigateur internet (<http://localhost:4200>):



```
login

username: user1
password: pwd1
roles: user
login

message: valeurs saisies={"username":"user1","password":"pwd1","roles":"user"}
```

NB : au sein d'un tp ultérieur , les styles css seront nettement améliorés.

Avec validateurs (partie intéressante du Tp) :

Du côté .ts , peaufiner l'initialisation de **loginForm** en **plaçant plein de contraintes de saisies** dans la seconde partie de `= form(this.loginModel, (schemaPath) => { ... }) ;`

Par exemple :

- les champs username et password ne devront pas être laissés vides (required)
- le password devra avoir une longueur minimale de 3 caractères
- le username devra commencer par un caractère alphabétique (pas par un chiffre)

Pour prendre en compte ces contraintes de saisies , on pourra dans un premier temps :

- ajouter `[disabled]="loginForm().invalid()"` sur le bouton login
- ajouter un second bouton "validate" servant à déclencher la construction d'un message de validation (ex : `<input type="button" value="validate" [hidden]="loginForm().valid()" (click)="onValidateLoginForm()" />`)

avec du côté .ts un code de ce genre :

```
onValidateLoginForm(){
  this.message="";
  if(/*this.loginForm.username().touched() &&*/ this.loginForm.username().invalid()){
    this.ok=false;
    for(let e of this.loginForm.username().errors())
      this.message += ` ${e.message} `
  }
  if(...) { ... }
}
```

Nouveau comportement attendu (à peu près):

username:	1a
password:	pp
roles:	user

validate login

message: username must start by letter , at least 2 characters password length must be at least 3

username:	user1
password:	pwd1
roles:	user

login

message: valeurs saisies={"username":"user1","password":"pwd1","roles":"user"}

Amélioration de la prise en compte des contrôles de saisies :

Au lieu de construire et afficher un message d'erreur global lorsque l'on clique sur "validate" , on peut dynamiquement reconstruire et ré-afficher des petits messages ne concernant qu'un champ de saisie :

login

username:	1a	username must start by letter , at least 2 characters
password:	a	password length must be at least 3
roles:		

message:

login

username:	user1
password:	pwd1
roles:	user

message: valeurs saisies={"username":"user1","password":"pwd1","roles":"user"}

Type de code possible du côté .ts :

```
usernameError=computed(
  ()=> this.loginForm.username().errors().map(e=>e.message).join(" "));

isLoginFieldValid(fieldName:string ){
  switch(fieldName){
    case "username": return ! this.loginForm.username().invalid();
    case "password": return ! this.loginForm.password().invalid();
    case "roles": return ! this.loginForm.roles().invalid();
    default : return false;
  }
}

classForLoginField(fieldName:string) {
  let v= this.isLoginFieldValid(fieldName);
  return {
    'ng-valid': v,
    'ng-invalid': !v,
  }
}
```

Type de code possible du coté .html :

```
<label class="basic-align">username:</label>
<input [formField]="loginForm.username" [ngClass]="classForLoginField('username')" />
<span class="text-red-600">{{usernameError()}}</span>
```

Type de code possible du coté .css :

```
input.ng-valid {
  border-left: 5px solid #42A948; /* green */
}
input.ng-invalid {
  border-left: 5px solid #a94442; /* red */
}
```

2. TD facultatif : intégration de tailwind-css

2.1. Intégration de tailwind-css dans angular (si nécessaire)

1) ajouter tailwindcss et postcss dans le projet angular

```
npm install --save-dev tailwindcss @tailwindcss/postcss postcss
```

2) ajouter le fichier **.postcssrc.json** à la racine du projet angular avec ce contenu :

```
{
  "plugins": {
    "@tailwindcss/postcss": {}
  }
}
```

3) importer tailwindcss dans **src/styles.css**

```
/* @import 'tailwindcss'; */
@import 'tailwindcss/theme';
@import 'tailwindcss/utilities';
```

2.2. Formulaire "responsive" avec champs bien alignés:

On va maintenant terminer ce TD en **ajoutant** les classes de styles de tailwind css (ex: "grid" , "w-full" , ...) au sein de **login.component.html** pour obtenir un formulaire "responsive" avec des **champs bien alignés**:

```
<h3>login</h3>
<form novalidate class="grid grid-cols-1 md:grid-cols-2 gap-4">
  <div>
    <label class="block text-sm">username:</label>
    <input [formField]="loginForm.username" class="w-full" [ngClass]="classForLoginField('username')" />
    <span class="text-red-600">{{usernameError()}}</span>
  </div>
  ...
</form>
```

Sur écran large (affichage sur 2 colonnes) :

login

username:	user1	password:	
		password is required	
roles:			
login			
message:			

sur écran étroit (ex : smartphone) , affichage sur 1 colonne :

login

username:	user1
password:	pwd1
roles:	
login	
message: valeurs saisies={"username":"user1","password":"pwd1","roles":""}	

NB: il est possible de paramétrer facilement une grille css sans tailwind.
Tailwind n'est qu'un framework facultatif mais néanmoins assez pratique .

3. TP input() sur composant header

Ajouter (via **input()**) un titre paramétrable au sein du composant HeaderComponent

Afficher la valeur de **{{titre()}}** dans header.component.html

Au sein de app.component.html , donner une valeur fixe au titre paramétrable de <app-header>

Tester le tout

Donner ensuite une valeur dynamique au titre paramétrable de <app-header> en demandant une copie de la valeur de AppComponent.title .

Tester le nouveau comportement.

my-app welcome basic

welcome



footer , date=13/09/2024

4. TP simpleSelector via output() puis model()

- (idéalement dans **src/app/common/component**) générer un nouveau composant réutilisable appelé "**SimpleSelector**" et intégrer le quelque part (par exemple au sein de **basic.component.html**)
- coder une première version de ce composant (avec input() et output()) de manière à ce que l'on puisse paramétrer :
 - * un *title* variable
 - * une liste variable *values* de valeurs (de type string[]) à choisiret que l'on puisse récupérer la valeur choisie sous la forme d'un événement *choix*

Exemple de look souhaité pour le composant "SimpleSelector" ici en fond gris:

choix couleur

- red
- **green**
- blue

couleurChoisie = green

Exemple d'utilisation souhaitée de ce composant depuis un parent :

```
<app-simple-selector  
  title="choix couleur"  
  [values]="[ 'red', 'green', 'blue' ]"  
  (choix)="onChoixCouleur(.....)"></app-simple-selector> <br/>  
couleurChoisie = {{couleurChoisie}}
```

Suite facultative du Tp :

- Sauvegarder des copies de simple-selector.component.html et .ts (ex : v1.txt)
- Puis au sein de simple-selector.component.ts , transformer *choix* de output() en **model()** de manière à ce que le parent puisse (s'il en a envie) fixer un choix par défaut (ex : 'green')
- rendre tout cela cohérent et tester le nouveau comportement (choix par défaut paramétrable) dans une version adaptée du composant parent (après une éventuelle copie de l'ancienne version en commentaires)

5. TD projection dans composant réutilisable

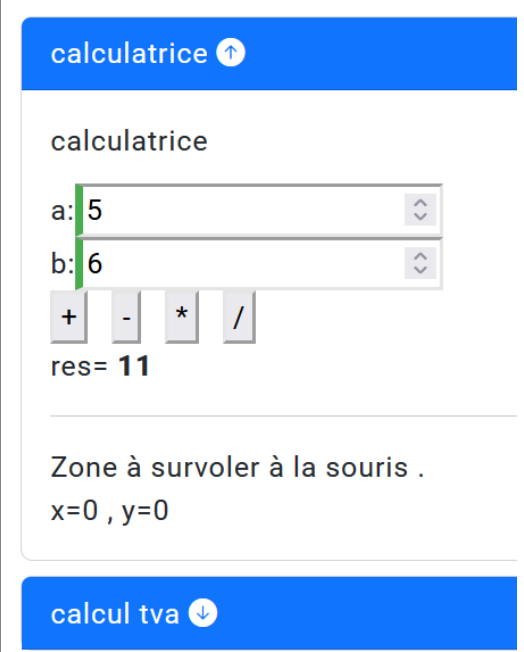
Préparer l'arborescence src/app/**common/component** et se placer dans ce répertoire .

```
ng g component toggle-panel --type component
```

Coder le contenu du composant **toggle-panel** en s'inspirant du code du support de cours (chapitre "composants et modules / approfondissements" et sous partie "projection d'éléments imbriquées").

avec `<ng-content></ng-content>` et avec ou sans l'utilisation de **tailwind-css** .

Ajuster le contenu de basic.component.html de façon à ce que les composants "calculatrice" et "tva" soient imbriqués dans des panneaux déployables/rétractables :

<pre><app-toggle-panel title="calculatrice"> <app-calculatrice></app-calculatrice> </app-toggle-panel> <app-toggle-panel title="calcul tva"> <app-tva></app-tva> </app-toggle-panel></pre>	
---	--

6. TD (facultatif) : onglet de @angular/material

Intégration de l'extension @angular/material au sein du projet angular :

ng add @angular/material

>>> Choisir un des thèmes proposés et accepter les animations

Importation du module "MatTabsModule" (onglets de material) :

Ajouter dans src/app/app.module.ts ou bien dans src/app/basic/basic.component.ts

```
import {MatTabsModule} from '@angular/material/tabs';  
et  
imports: [ ..., MatTabsModule, ],...
```

Utilisation possible au sein de basic.component.html :

```
<mat-tab-group>  
  <mat-tab label="calculatrice">  
    <app-calculatrice></app-calculatrice>  
  </mat-tab>  
  <mat-tab label="calcul tva">  
    <app-tva></app-tva>  
  </mat-tab>  
</mat-tab-group>  
<hr/>
```

calculatrice

calcul tva

calcul tva

ht:

tauxTva(%):

Un redémarrage de ng serve sera peut être nécessaire ...

7. TP (facultatif) directive "border-over"

Au sein du composant "welcome" , placer une série de 3 ou 4 petites images.

On va maintenant coder et utiliser une **directive personnalisée "border-over"** qui va automatiquement mettre en évidence une des images survolées (en ajoutant une bordure par défaut rouge ou bien d'une couleur paramétrable).

Indications :

créer si besoin le sous répertoire src/app/common/directive

ng g directive border-over